Hyperflex 🚀 elastic

Elasticsearch as the Memory Layer for AI Agents

The jump from Demo AI to Production AI with Elasticsearch managed memory

Vishal Rathod
Presenter
Co-founder/CTO - Hyperflex

Hyperflex

Customer Information and Agent Details

Customer Base

- → Retail
- → E-Commerce
- → Medical Suppliers
- → Staffing
- → Consulting

Metrics

- → Demo: 46
- → Shipped: 28
- → In Production after 6 months: 8
- → ROI: ???

Hyperflex

This gap in demo AI to production AI is what we will be talking about today

Transitioning from a demo to production-grade AI requires a memory layer that evolves beyond simple vector similarity. In tis session we will explores how to use Elasticsearch as a high-performance "brain" for autonomous agents, implementing a sophisticated procedural, episodic, and semantic memory architecture. We'll dive into how Elasticsearch's document-level security and hybrid search solve selective memory isolation—a critical production challenge involving metadata and permissions that pure vector databases cannot handle. Join us to learn the architectural blueprint for turning forgetful prototypes into reliable, distributed, and context-aware agentic systems.

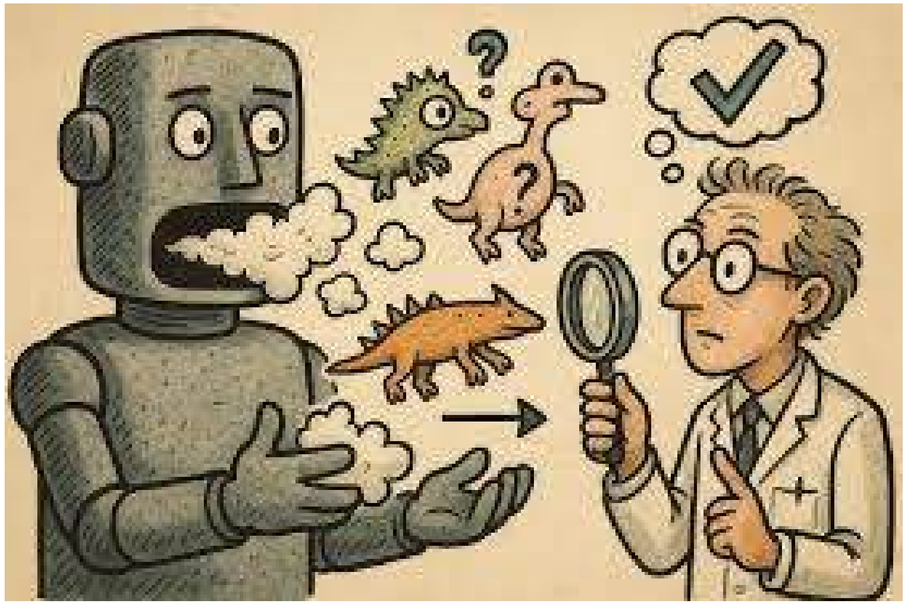
Problem :

→ LLMs are Stateless

→ Context alone cant solve smart retrieval

→ Context Poisoning

→ Compartmentalised Memory



Approaches :

- Use case based Isolation
- Context Awareness
- Memory clearance without impacting others
- History Recharging when needed
- Human friendly approach
- Weighted Retrieval

Solution:

Hyperflex

CoALA

Cognitive Architectures for Language Agents

https://arxiv.org/html/2309.02427v3#S1

1 Introduction

2 Background: From Strings to Symbolic AGI

3 Connections between Language Models and Production Systems

4 Cognitive Architectures for Language Agents (CoALA): A Conceptual Framework

5 Case Studies

6 Actionable Insights

7 Discussion

8 Conclusion

Cognitive Architectures for Language Agents

Theodore R. Sumers[1] **Shunyu Yao**[1] **Karthik Narasimhan** **Thomas L. Griffiths**

Princeton University

{sumers, shunyuy, karthikn, tomg}@princeton.edu

Abstract

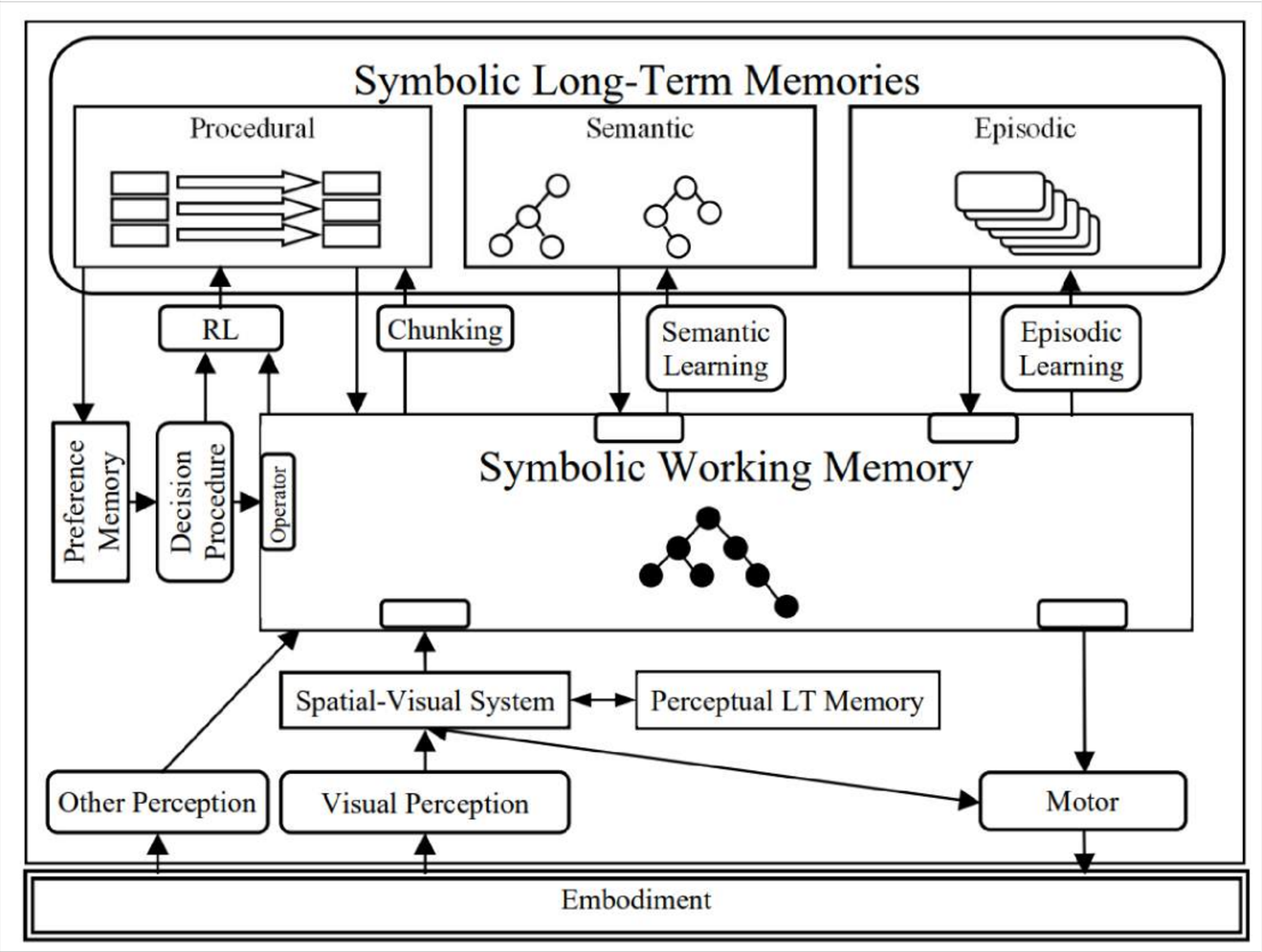
Recent efforts have augmented large language models (LLMs) with external resources (e.g., the Internet) or internal control flows (e.g., prompt chaining) for tasks requiring grounding or reasoning, leading to a new class of *language agents*. While these agents have achieved substantial empirical success, we lack a framework to organize existing agents and plan future developments. In this paper, we draw on the rich history of cognitive science and symbolic artificial intelligence to propose Cognitive Architectures for Language Agents (CoALA). CoALA describes a language agent with modular memory components, a structured action space to interact with internal memory and external environments, and a generalized decision-making process to choose actions. We use CoALA to *retrospectively* survey and organize a large body of recent work, and *prospectively* identify actionable directions towards more capable agents. Taken together, CoALA contextualizes today's language agents within the broader history of AI and outlines a path towards language-based general intelligence.

1 Introduction

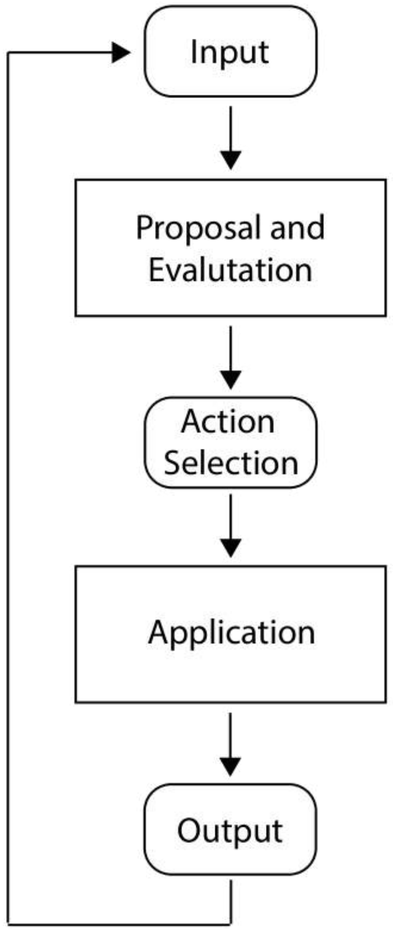
Language agents (Weng, 2023; Wang et al., 2023b; Xi et al., 2023; Yao and Narasimhan, 2023) are an emerging class of artifical intelligence (AI) systems that use large language models (LLMs; Vaswani et al., 2017; Brown et al., 2020; Devlin et al., 2019; OpenAI, 2023a) to interact with the world. They apply the latest advances in LLMs to the existing field of agent design. (Russell

Cognitive Architectures for Language Agents

A



B



Procedural Memory:
The HOW: agent's operating system or its set of "reflexes."

Key Functions

It governs the "if/then" logic of the system, specifically:

- Memory Management: Deciding the exact moment to save or recall a piece of information.
- Information Synthesis: Knowing how to condense long dialogues into concise summaries.
- Skills Integration: Determining which tools to trigger and how to execute them.

Implementation

In this architecture, procedural memory is functional logic, not a static record:

- Location: It resides in the source code and prompts, acting as the engine.
- The Elastic Relationship: Procedural memory is not stored in Elastic; rather, it uses Elastic as a resource to fetch the data it needs to function.

The Bottom Line: Procedural memory defines the process, not the payload.

Episodic Memory:
What: unique experiences and individual preferences rather than general facts.

Examples:

“Peter’s birthday is tomorrow and he wants steak.”

“Janice has a report due at 9 am.”

Episodic Memory:
What: unique experiences and individual preferences rather than general facts.

Implementation in the Architecture

In this system, episodic memory is treated as a searchable database of experiences:

- **Storage**: Each memory is saved as an individual document within Elastic.
- **Metadata Tagging**: To maintain accuracy, every entry is labeled with technical context, such as the User, Role, Timestamp, and whether it is an "Innie" or "Outie" (internal vs. external) memory.
- **Selective Retrieval**: The agent doesn't dump all memories at once; it carefully filters its search based on who is currently interacting with it and the current situation.

The Bottom Line: Episodic memory provides the personal context that allows an agent to feel consistent and attentive to individual users.

Semantic Memory:
Repository for ground truths, rules, and general knowledge.

Examples:

Company policies, math formulas, coding syntax, or the fact that "there are 12 months"

Semantic Memory:
Repository for ground truths, rules, and general knowledge.

Implementation in the Architecture

In a technical system, semantic memory acts as the "source of truth":

- **Documentation**: Items like company handbooks, standard operating procedures (SOPs), and general knowledge bases are ingested here.
- **Accessibility**: Because this information is considered "world truth," it is often shared across different user roles or contexts to ensure the agent's logic remains consistent.
- **Function**: It provides the grounding necessary for the agent to provide accurate, factual answers that aren't influenced by personal bias or fleeting experiences.

The Bottom Line: Semantic memory provides the shared reality that allows an agent to be factually reliable.

Hyperflex

How Elastic can help implement CoALA

- → Document level Security, Memory Isolation
- → Agent Tools, Skills
- → Smart Retrieval: ELSER (Weighted Retrieval)

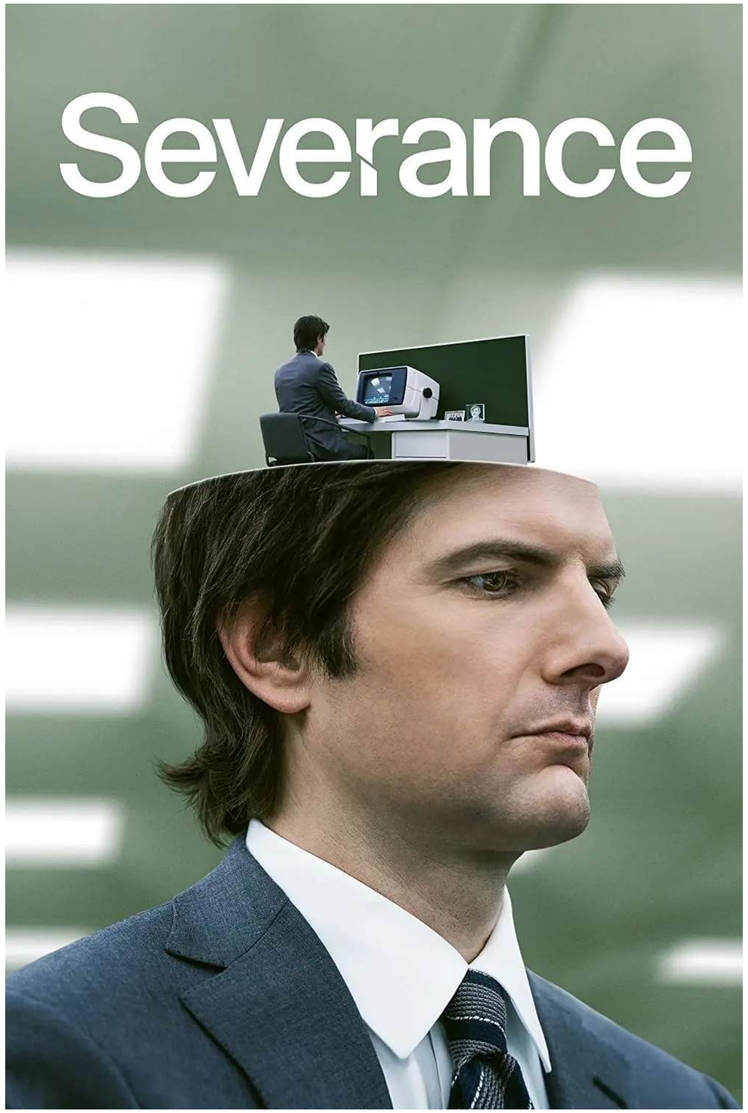
Elasticsearch is no longer just a search engine, it is the cognitive infrastructure layer for production AI agents. The three types of agent memory map directly to Elastic's native capabilities, and that mapping is what separates brittle RAG demos from reliable production systems.

The fix is not a bigger context window. The fix is external memory, and Elastic is the best database for it, because it's simultaneously a vector DB, a metadata store, and a security layer.

The RRF (Reciprocal Rank Fusion) retriever merges both result lists. This is the query your agent runs every time it needs to inject context into the LLM.

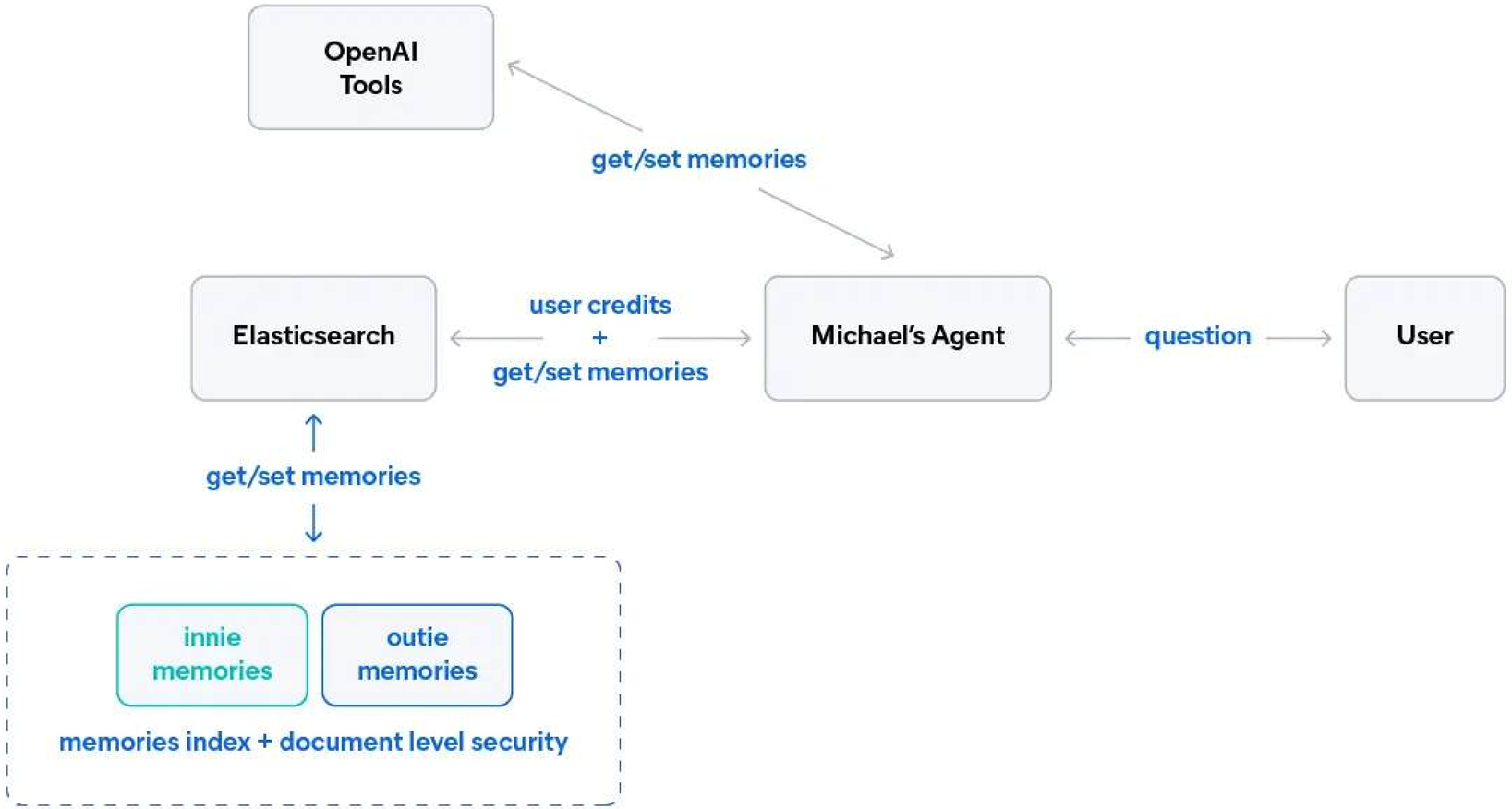
Hyperflex

Demo:



Hyperflex

Demo: Architecture



Poison type	What happens · Elasticsearch defence
Context rot	Outdated docs retrieved as current truth → Temporal filters with date range queries
Context overflow	Too much retrieved text drowns relevant signal → Selective retrieval with metadata pre-filters
Conflicting info	Multiple versions/configs confuse the model → Metadata boosting with deployment_type/version fields
Semantic noise	Vectorially similar but contextually wrong docs → Minimum score thresholds on semantic queries
Memory bleed	User A's episodic memories surface for User B → Document Level Security (DLS)

Layer	Components
User layer	User sends message → Application code (procedural memory) receives it
Retrieval layer	ES hybrid search on memories index (RRF: BM25 + ELSER) · DLS applies user role automatically
Injection layer	Top-3 episodic memories + top-2 semantic facts injected into LLM system message
LLM layer	OpenAI GPT-4o (or any model) generates response grounded in retrieved context
Write-back layer	Agent stores new memory document to ES · ELSER auto-indexes it · Available next session

Get the Slides and Demo Video on Demand:
"Elastic as the Memory Layer for AI Agents On-Demand"
and other resources

Hyperflex

Learn more : hyperflex.co